```
import numpy as np
import cv2
import matplotlib.pyplot as plt

def evaluate_fitness(threshold, histogram):
    """Calculates standard 2-class Otsu between-class variance for 1 threshold."""
    total_pixels = np.sum(histogram)
    if total_pixels == 0:
        return 0.0

    intensities = np.arange(256)
    t = int(np.clip(threshold, 1, 254))

    w0 = np.sum(histogram[:t+1]) / total_pixels
    w1 = np.sum(histogram[t+1:]) / total_pixels

    if w0 == 0 or w1 == 0:
        return 0.0

    u0 = np.sum(intensities[:t+1] * histogram[:t+1]) / (np.sum(histogram[:t+1]) + 1e-10)
    u1 = np.sum(intensities[t+1:] * histogram[t+1:]) / (np.sum(histogram[t+1:]) + 1e-10)

    between_class_variance = w0 * w1 * ((u0 - u1) ** 2)
    return between_class_variance

def single_threshold_aco(image, num_ants=30, max_iter=60, evaporation_rate=0.15, Q=25.0):
    """Finds 1 optimal intensity threshold using Ant Colony Optimization."""
    histogram, _ = np.histogram(image, bins=256, range=(0, 256))
    np.random.seed(42)
    intensities = np.arange(256)

    # Pheromone trail across intensities 0 to 255
    pheromones = np.ones(256)

    # Start with an initial baseline threshold
    gbest_threshold = 50
    gbest_fitness = evaluate_fitness(gbest_threshold, histogram)

    print(f"Initialization | Global Best Threshold: {gbest_threshold:03d} | Fitness Score: {gbest_fitness:.4f}")

    for iteration in range(1, max_iter + 1):
        # Convert pheromone levels into selection probabilities
        probabilities = pheromones / np.sum(pheromones)

        # Ants pick candidate thresholds
        ant_thresholds = np.random.choice(intensities, size=num_ants, p=probabilities)
        fitnesses = np.zeros(num_ants)

        for i in range(num_ants):
            t = int(ant_thresholds[i])
            fit = evaluate_fitness(t, histogram)
            fitnesses[i] = fit

            # Update global best threshold
            if fit > gbest_fitness:
                gbest_fitness = fit
                gbest_threshold = t

        # Pheromone Evaporation
        pheromones *= (1.0 - evaporation_rate)

        # Pheromone Deposit with Gaussian neighborhood spread
        max_fit = np.max(fitnesses) + 1e-10
        for i in range(num_ants):
            if fitnesses[i] > 0:
                t = int(ant_thresholds[i])
                deposit = Q * (fitnesses[i] / max_fit)
                pheromones += deposit * np.exp(-0.5 * ((intensities - t) / 4.0) ** 2)

        # Print milestone updates
        if iteration in [1, 10, 20, 30, 40, 50, max_iter]:
            print(f"Iteration {iteration:02d}   | Global Best Threshold: {gbest_threshold:03d} | Fitness Score: {gbest_fitness:.4

    return gbest_threshold, histogram
```

```
if __name__ == "__main__":
    # 1. Load your UML diagram in grayscale
    image_path = "/content/sample_data/70dij1oq4bke1.jpeg"
    img = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)

    if img is None:
        raise FileNotFoundError(f"Could not find ''. Place your image in the same directory.")

    print("[SYSTEM] Computing image histogram probabilities... Done.")
    print("[SYSTEM] Initializing 30 ants and pheromone trails across domain [0, 255]...\n")

    # 2. Run 1-Threshold ACO
    optimal_t, histogram = single_threshold_aco(img)
    print(f"\nFinal Optimal Threshold: {optimal_t}")

    # 3. Apply binary thresholding (<= optimal_t becomes 0, > optimal_t becomes 255)
    _, segmented_img = cv2.threshold(img, optimal_t, 255, cv2.THRESH_BINARY)
    output_path = "uml_segmented_binary.png"
    cv2.imwrite(output_path, segmented_img)
    print(f"[SYSTEM] Binary segmented image saved to '{output_path}'.")

    # 4. Display Original Image, Histogram, and Binary Segmented Result
    plt.figure(figsize=(15, 5))

    plt.subplot(1, 3, 1)
    plt.imshow(img, cmap="gray")
    plt.title("Original Grayscale UML Diagram")
    plt.axis("off")

    plt.subplot(1, 3, 2)
    plt.plot(histogram, color="black")
    plt.axvline(optimal_t, color="red", linestyle="--", label=f"Threshold T = {optimal_t}")
    plt.title("Pixel Intensity Histogram & Optimal Threshold")
    plt.xlabel("Pixel Intensity (0-255)")
    plt.ylabel("Pixel Count")
    plt.yscale("log")
    plt.legend()

    plt.subplot(1, 3, 3)
    plt.imshow(segmented_img, cmap="gray")
    plt.title(f"Binary ACO Segmented (T = {optimal_t})")
    plt.axis("off")

    plt.tight_layout()
    plt.savefig("aco_binary_comparison.png", dpi=150)
    plt.show()
```

```
[SYSTEM] Computing image histogram probabilities... Done.
[SYSTEM] Initializing 30 ants and pheromone trails across domain [0, 255]...

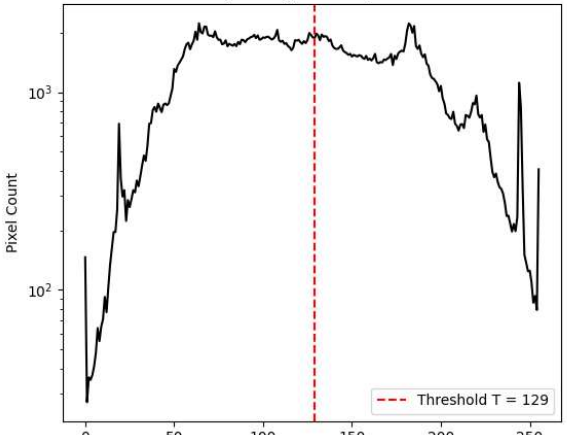
Initialization | Global Best Threshold: 050 | Fitness Score: 601.1213
Iteration 01   | Global Best Threshold: 131 | Fitness Score: 2095.6814
Iteration 10   | Global Best Threshold: 129 | Fitness Score: 2097.0760
Iteration 20   | Global Best Threshold: 129 | Fitness Score: 2097.0760
Iteration 30   | Global Best Threshold: 129 | Fitness Score: 2097.0760
Iteration 40   | Global Best Threshold: 129 | Fitness Score: 2097.0760
Iteration 50   | Global Best Threshold: 129 | Fitness Score: 2097.0760
Iteration 60   | Global Best Threshold: 129 | Fitness Score: 2097.0760

Final Optimal Threshold: 129
[SYSTEM] Binary segmented image saved to 'uml_segmented_binary.png'.
```

Original Grayscale UML Diagram



Pixel Intensity Histogram & Optimal Threshold



Binary ACO Segmented (T = 129)

